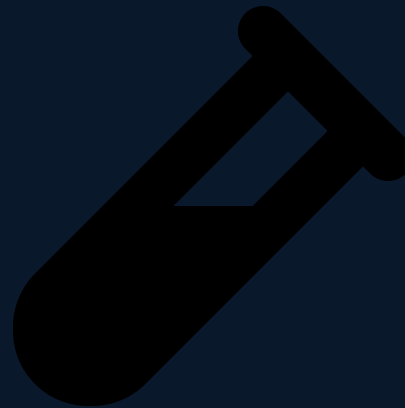


PORTADA

Análisis y Diseño de Sistemas II

Semana 16 — Pruebas Unitarias de Software: Fundamentos y Frameworks

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

**Refactorizar sin pruebas
es apostar a que nada se rompió**

TRANSICIÓN

De la advertencia de la semana pasada, a la herramienta que la resuelve

La semana pasada se cerró con una advertencia pendiente: refactorizar sin una red de pruebas automatizadas es peligroso, porque nada garantiza que el comportamiento externo del código siga siendo el mismo después del cambio. Hoy se abre la **semana 16, cierre de la Unidad 6**, con esa herramienta: las **pruebas unitarias de software**.

Esta sesión cubre los fundamentos — qué es una prueba unitaria, cómo se estructura, qué la hace buena, y qué frameworks existen por lenguaje. El miércoles se profundiza en **TDD** (Test-Driven Development), y el viernes se cierra la unidad completa con un laboratorio práctico sobre el proyecto de curso.

CONCEPTO

Qué es una prueba unitaria

Una **prueba unitaria** (unit test) es un fragmento de código que verifica automáticamente que la **unidad más pequeña de código** con comportamiento propio —típicamente un método o una función— se comporta como se espera, de forma aislada del resto del sistema.

Cuatro características la distinguen de simplemente "probar la aplicación a mano":

- **Aísla** la unidad bajo prueba de sus dependencias externas (base de datos, red, sistema de archivos).
- Es **rápida** — se puede ejecutar cientos o miles de veces al día.
- Es **repetible** — el mismo resultado, sin importar cuántas veces se ejecute ni en qué máquina.
- Es **automatizada** — no requiere que una persona verifique el resultado manualmente.

CASO REAL

El origen de JUnit: Kent Beck y Erich Gamma, 1997

Uno de los frameworks de pruebas más influyentes de la historia, **JUnit**, nació casi por accidente. En 1997, **Kent Beck** (creador de Extreme Programming y de TDD) y **Erich Gamma** (uno de los cuatro autores del libro de patrones GoF que vieron en la Unidad 5) lo escribieron durante un vuelo transatlántico, como ejercicio para que Beck le enseñara a Gamma a programar en Java.

JUnit popularizó el patrón **xUnit** — una familia de frameworks de pruebas con la misma filosofía adaptada a distintos lenguajes (JUnit para Java, NUnit para .NET, PyUnit/pytest para Python) — y ayudó a que las pruebas automatizadas pasaran de ser una práctica minoritaria a un estándar de la industria.

CONCEPTO

La pirámide de pruebas

No todo el software se verifica solo con pruebas unitarias. La **pirámide de pruebas** es un modelo que organiza los distintos niveles de prueba según su alcance y costo:

- **Pruebas unitarias** (base, la capa más ancha): verifican una función o clase aislada. Rápidas y baratas de escribir y ejecutar — deberían ser la mayoría.
- **Pruebas de integración** (medio): verifican que varios componentes trabajen bien juntos (ej. un servicio con su base de datos real).
- **Pruebas end-to-end / E2E** (punta, la más angosta): verifican el sistema completo desde la perspectiva del usuario, a través de la interfaz real.



CONCEPTO

Por qué esa proporción, y no otra

La forma de pirámide no es arbitraria: refleja una relación inversa entre **alcance** y **velocidad/costo**. Mientras más "arriba" está una prueba, más partes reales del sistema involucra —y por eso es más lenta, más costosa de mantener y más frágil ante cambios que no tienen relación directa con lo que se quiere verificar.

Una prueba E2E puede tardar segundos o minutos y romperse por un cambio de interfaz sin relación con la lógica que prueba; una prueba unitaria bien aislada tarda milisegundos y solo se rompe si la lógica que prueba realmente cambió. Por eso conviene tener **muchas** pruebas unitarias baratas, y **pocas** pruebas E2E costosas, reservadas para los flujos más críticos.

CONCEPTO

Estructura AAA (Arrange-Act-Assert)

Casi toda prueba unitaria bien escrita sigue el mismo patrón de tres pasos, conocido como **AAA** (o su equivalente en lenguaje de negocio, **Given-When-Then**):

- **Arrange (Given)**: preparar los datos y el estado necesario para la prueba.
- **Act (When)**: ejecutar la acción o el método que se quiere probar.
- **Assert (Then)**: verificar que el resultado obtenido es el esperado.

```
def test_calcula_descuento_para_cliente_premium():  
    # Arrange  
    cliente = Cliente(tipo="premium")  
    carrito = Carrito(subtotal=100)  
    # Act  
    total = calcular_total(carrito, cliente)  
    # Assert  
    assert total == 90
```


CONCEPTO

Qué hace buena una prueba unitaria: FIRST

FIRST es un acrónimo que resume las cinco propiedades que debería tener toda buena prueba unitaria:

Letra	Propiedad	Significa
F	Fast (rápida)	Se ejecuta en milisegundos, no segundos.
I	Independent (independiente)	No depende del resultado ni del orden de otra prueba.
R	Repeatable (repetible)	Mismo resultado en cualquier entorno, sin depender de red o fecha del sistema.
S	Self-validating (autoverificable)	Da un resultado claro de aprobado/fallido, sin revisión manual.
T	Timely (oportuna)	Se escribe junto con el código, no meses después.



CONCEPTO

Frameworks comunes por lenguaje

Casi todo lenguaje moderno tiene uno o varios frameworks de pruebas maduros, siguiendo el modelo xUnit que originó JUnit:

Lenguaje / plataforma	Framework
Java	JUnit 5
.NET (C#)	xUnit.net / NUnit
Python	pytest
JavaScript / TypeScript	Jest

Cada equipo de la Fábrica Escuela debería usar el framework correspondiente a su propio stack tecnológico — es el que se usa en el laboratorio del viernes.

CONTRAEJEMPLO

La misma prueba en tres frameworks

```
// JUnit 5 (Java)
@Test
void debeCalcularDescuentoParaClientePremium() {
    assertEquals(90, calcularTotal(carrito, clientePremium));
}

// Jest (JavaScript)
test("calcula descuento para cliente premium", () => {
    expect(calcularTotal(carrito, clientePremium)).toBe(90);
});

// pytest (Python)
def test_calcula_descuento_para_cliente_premium():
    assert calcular_total(carrito, cliente_premium) == 90
```

La sintaxis cambia según el lenguaje, pero la estructura **AAA** y la intención son idénticas en los tres — por eso aprender bien un framework hace mucho más fácil aprender cualquier otro de la familia xUnit.

CONCEPTO

Mocks, stubs y dobles de prueba

Para que una prueba unitaria esté realmente **aislada** (la "I" de FIRST) e independiente del entorno, no puede depender de una base de datos real, una API externa o el reloj del sistema. La solución es reemplazar esas dependencias por objetos falsos, llamados en conjunto **dobles de prueba** (test doubles):

- **Stub:** un objeto falso que devuelve respuestas predefinidas, sin lógica real — ej. un stub de repositorio que siempre devuelve el mismo usuario de prueba.
- **Mock:** similar a un stub, pero además **verifica** que fue llamado correctamente (con qué argumentos, cuántas veces) — útil para confirmar que se envió una notificación sin enviarla de verdad.

Cuándo usar un doble en vez del objeto real: siempre que la dependencia real sea lenta, no determinista (red, fecha actual), o costosa de configurar en un entorno de pruebas.

INTERACCIÓN

¿Unitaria, integración o E2E?

En el chat, para cada prueba indiquen en qué nivel de la pirámide la ubicarían:

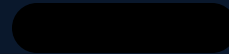
1. Verificar que `calcularDescuento(cliente, monto)` devuelve el valor correcto para un cliente premium, sin tocar base de datos.
2. Verificar que, al hacer clic en "Comprar" en el navegador, el pedido aparece completo en la pantalla de confirmación.
3. Verificar que el servicio de pedidos realmente guarda y recupera un registro en la base de datos de pruebas.

5 minutos · respondan en el chat

SÍNTESIS

Ideas clave de la sesión

1. Una prueba unitaria aísla la unidad más pequeña de código, y es rápida, repetible y automatizada.
2. La pirámide de pruebas prioriza muchas pruebas unitarias baratas sobre pocas pruebas E2E costosas.
3. AAA (Arrange-Act-Assert) es la estructura estándar de cualquier prueba bien escrita.
4. FIRST resume qué hace buena una prueba: rápida, independiente, repetible, autoverificable y oportuna.
5. Los dobles de prueba (mocks, stubs) aíslan la unidad de sus dependencias externas lentas o no deterministas.



CIERRE

Antes de la próxima sesión

Identifiquen qué framework de pruebas les corresponde según el stack tecnológico de su proyecto (JUnit 5, xUnit/NUnit, pytest o Jest) y confirmen que está instalado y configurado en su repositorio. El miércoles se ve **TDD (Test-Driven Development)**: el ciclo Red-Green-Refactor y las buenas prácticas para escribir pruebas que realmente ayuden al diseño.

